

Week 7 周回顾：工程化闭环（可观测 + 可靠性 + 安全 → 项目整合）

本周目标：把前 6 周“能跑的 demo”升级成“能上生产的企业级系统”。这是**工程化与玩具项目**的分水岭，也是面试必问“**你怎么监控 Agent 性能？**”的答案来源。核心收获：“**我不仅能写 Agent，还能监控它（LangFuse）、扛住它（重试/队列/超时/熔断）、保护它（权限沙箱）**”——这三点正是区分中级和高级工程师的标尺。本周主线叙事（面试杀手锏）：**功能实现 → 可观测 → 可靠性 → 安全 → 项目整合**。

1. Week 7 周总结表（Day 1-7 一览）

天	主题	完成内容	产出文件	状态
Day 1	LangFuse 接入（可观测）	@observe() 包装 traced_chat / traced_chat_with_tools / traced_retrieve 三个观测点； 双后端透明切换 （LangFuse ↔ 本地 JSON 日志，无外网优雅降级）；demo_tracing() 演示追踪链路	app/observability/tracing.py + langfuse_notes.md	✅ 已完成
Day 2	LangFuse Evaluation	三指标 score_faithfulness / score_answer_relevance / score_context_precision（LLM-as-Judge + 中文 2-gram 规则兜底双通道 ）；evaluate_rag 聚合 + send_scores_to_langfuse 写回 feedback loop；demo_eval 5 组样本验证掉分诊断	app/observability/eval.py + langfuse_notes.md	✅ 已完成
Day 3	Harness 概念体系	Harness 四件套架构图 + 组件职责对照表 + 与已有代码映射（ConversationMemory / HITL / max_rounds）；找出最缺的两块（重试 + 调度）	harness_notes.md	✅ 已完成
Day 4	可靠性实战	retry_with_backoff（1s→2s→4s + jitter）/ TaskQueue（asyncio.Queue 生产-消费）/ run_with_timeout + run_with_timeout_sync（30s 超时降级）/ CircuitBreaker（加分项：closed→open→half_open 状态机）；test_reliability.py 20 项测试全 PASS	app/agent/reliability.py + test_reliability.py	✅ 已完成
Day 5	权限沙箱概念	Agent 安全三原则（工具权限最小化 / HITL / 沙箱隔离）+ 工具风险分级表	security_notes.md	✅ 已

天	主题	完成内容	产出文件	状态
		(只读放行 / 写审批 / 高危禁止) + 三道防线安全边界图		完成
Day 6	项目整合	FastAPI 12 条路由 (新增 /rag/supervisor /rag/router) ; _safe_answer() 统一可靠性调用链 (CircuitBreaker→retry→30s 超时降级) + _trace_span() 可观测接入点 (ENABLE_TRACING / ENABLE_EVAL 开关) ; README 架构更新	app/main.py + README.md	✅ 已完成
Day 7	周回顾 + 完整架构图	"企业级 Agent 系统全链路"架构图 + 3 面试题精讲 + 本周必会手写的代码清单 + Git/自检清单	本文件	✅ 已完成

一句话本周：Day 1-2 解决"看不见"（可观测），Day 3-4 解决"扛不住"（可靠性），Day 5 解决"不安全"（权限沙箱），Day 6 把三件套统一装进 FastAPI 形成完整系统，Day 7 串成"工程化闭环"主线。

2. "工程化闭环"叙事线（本周核心主线）

2.1 完整链路图



一句话主线：前 6 周你把 RAG 和 Agent 的**功能**做完了（能检索、能推理、能调工具）；第 7 周不写任何新功能，而是给整套系统装上**工程化三件套**——**可观测**（看得见）、**可靠性**（扛得住）、**安全**（不失控），最后整合成一个能上生产的 FastAPI 服务。

工程化 = 可观测 + 可靠性 + 安全，把"能跑的 demo"变成"能上生产的企业系统"。面试讲这条线，能把 Week 4-7 全部内容串成一张图：功能（前 6 周）是"做什么"，工程化（本周）是"怎么让它可靠地、安全地、可观察地做"。

2.2 为什么需要工程化？（三个痛点驱动三件套）

痛点	表现	对应工程化组件	本周落地
● 看不见	用户抱怨"回答不对"，无法定位是 LLM / 检索 / 工具哪个环节出问题	可观测	LangFuse 追踪 3 观测点 + 评估 3 指标

痛点	表现	对应工程化组件	本周落地
● 扛不住	网络抖动 LLM 失败一次整个回答崩掉；多用户并发互相阻塞；请求卡死挂起	可靠性	重试 + 队列 + 超时 + 熔断
● 不安全	Agent 拿到工具就能乱调，误操作 / 恶意操作 / 数据泄露无法拦截	安全	白名单 + HITL + Docker 沙箱

3. 三大板块精讲（面试核心）

3.1 板块一：可观测（LangFuse 追踪 + 评估）

3.1.1 核心认知：追踪 vs 评估

追踪只告诉你“发生了什么”，评估才能告诉你“回答得好不好”。追踪 → 定位“慢在哪 / token 烧在哪”；评估 → 定位“质量差在哪个环节”。两者合起来形成 **feedback loop**（低分样本 → 反向优化检索 top_k / prompt）。

3.1.2 追踪 3 观测点 ↔ RAG 3 环节

观测点	观测什么	追踪函数	对应 RAG 环节	项目代码
LLM token 消耗	输入/输出 token、成本估算、模型名	<code>traced_chat_with_tools</code> （拿 <code>response.usage</code> ）	生成	<code>tracing.py:200</code>
LLM 输入输出/耗时	每次 LLM 调用耗时与输入输出	<code>traced_chat</code>	生成	<code>tracing.py:187</code>
检索耗时/命中	检索耗时、命中数量、命中片段	<code>traced_retrieve</code>	检索	<code>tracing.py:226</code>

关键设计（面试亮点）：双后端透明切换。 `tracing.py:46` 的 `_langfuse_configured()` 探测“库可 import + `LANGFUSE_*` 三变量齐备”→ 可用则用 `langfuse.decorators.observe`（后端 A），否则自动降级为自写的本地 JSON 日志装饰器 `_local_observe()`（后端 B, `tracing.py:112`）。**接口完全一致**——无外网时“追踪”的工程思路完整保留，只是输出载体从“看板”变成 `app.log` 的 `span.start` / `span.end` 记录。

3.1.3 评估 3 指标 ↔ RAG 3 环节（面试直接引用）

指标	判断逻辑	对应 RAG 环节	掉分时的修复方向
Faithfulness（忠实度）	回答里每个事实都能在 context 找到依据 → 1/0（无依据=编造）	生成环节（LLM 硬编）	检索不到时需“诚实回答：知识库中无此信息”兜底
Answer Relevance（回答相关性）	不看 context，只看“问题 vs 回答”是否对题 → 1/0（答非所问）	意图对齐环节（query 理解/prompt）	检查 query 理解与生成 prompt

指标	判断逻辑	对应 RAG 环节	掉分时的修复方向
Context Precision (上下文精准度)	正确答案是否出现在检索结果 靠前位置 → 1/0 (靠后/冗余)	检索环节 (Retriever)	调 top_k / 加重排

排查心法（面试话术）：先看哪个指标掉了 → 就能定位是哪个环节出问题。`eval.py:335` 的 `_diagnose()` 就是干这个的：`faithfulness=0` → 生成环节、`answer_relevance=0` → 意图对齐、`context_precision=0` → 检索环节。

双通道设计（与 tracing 双后端同一思路）：

- **通道 A「LLM-as-Judge」：**`eval.py:173` 的 `_llm_judge()` 用真实 DeepSeek 当裁判打分（优先）；
- **通道 B「规则兜底」：**`eval.py:122` 起的中文 **2-gram**（免分词）关键词覆盖率近似打分——LLM 不可用/超时也不崩，零成本任何环境可跑。`demo_eval` 里两条通道分数并列打印便于对照。

写回 feedback loop：`send_scores_to_langfuse()` 在 LangFuse 可用时 `lf.score()` 逐项写回 trace（看板看评估趋势），否则降级本地 JSON 日志——与 `tracing.py` 完全一致的降级风格。

3.1.4 追踪链路示意图（面试白板必备）



3.2 板块二：可靠性（Harness 四件套 + 三大硬技能 + 熔断器）

3.2.1 Harness 是什么？

一句话：Harness（智能体驾驭工程）= 让 Agent 在**生产环境稳定、安全、可控运行**的基础设施层。类比：**操作系统之于应用**——应用（Agent）只管做业务，操作系统（Harness）只管让应用跑得稳、跑得安全、跑得可控。

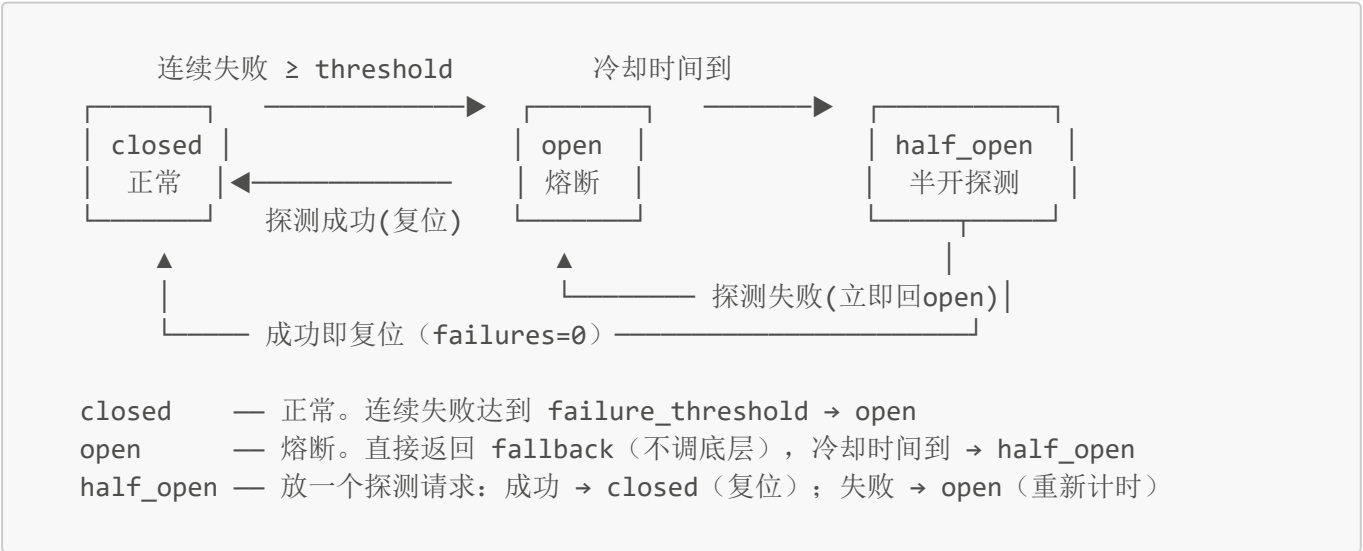
3.2.2 Harness 四件套 + 项目映射

组件	解决什么问题	一句话职责	项目已有代码
① Memory 管理	Agent 记住上下文，不越跑越偏	管理对话状态，喂"有界、分会话"的上下文	✅ ConversationMemory + SessionMemoryManager
② 权限沙箱	Agent 只能做"允许做的事"	给工具调用加"门卫"，拦截未授权动作	✅ HITL (research_agent_hitl.py) + dockerfile
③ 任务调度	多用户并发不打架，优先级可控	并发请求排进队列，逐个顺序执行	✅ 本周 TaskQueue (asyncio.Queue)
④ 异常重试	LLM/网络偶发失败不崩	失败自动重试 + 退避，超时熔断降级	✅ 本周 retry_with_backoff + run_with_timeout

3.2.3 三大硬技能（Day 4 手写核心）+ 加分项熔断器

组件	作用	关键代码位置	面试点
retry_with_backoff	LLM/网络偶发失败自动重试，1s→2s→4s + jitter	reliability.py:35	为什么加 jitter? 最后一次失败为什么直接抛?
TaskQueue	多用户请求排队，不互相阻塞	reliability.py:79	asyncio.Queue 生产-消费者; task_done()/join() 优雅关闭
run_with_timeout / run_with_timeout_sync	30s 超时返回降级话术，不挂死用户	reliability.py:137 / reliability.py:151	asyncio.wait_for 超时取消; 同步版守护线程不阻塞调用方
CircuitBreaker (加分)	连续失败 N 次 打开 → 快速失败 → 冷却 → 半开探测 → 关闭	reliability.py:182	重试管"偶发失败", 熔断管"持续故障"

3.2.4 熔断器状态机（面试白板必备）



代码对照: `CircuitBreaker.call()` 在 open 态且冷却未结束直接返回 `fallback` 快速失败 (`reliability.py:237`) ; 冷却结束切 `half_open` 放探测请求 (`reliability.py:243`) ; 探测成功复位 `closed` (`reliability.py:262`) 、探测失败立即回 open (`reliability.py:251`) 。测试验证完整状态序列 `closed→open→half_open→closed` (`test_reliability.py:212`) 。

3.2.5 测试验收 (20 项全 PASS)

`test_reliability.py` 不依赖 `pytest`, 用桩函数 (mock 抛异常 / mock sleep) 覆盖: 重试成功与全失败、jitter 随机性、队列按序执行与优雅关闭、`async/sync` 超时降级、熔断器完整状态机——**PASS=20, FAIL=0**。

3.3 板块三：安全 (三原则 + 工具风险分级 + 三道防线)

3.3.1 核心认知: Agent 越强，越危险

传统软件只能执行写死的逻辑; Agent 能自主决定"调哪个工具、传什么参数、做几次"。危险不只是恶意攻击, 更多是**无意误触发**——LLM 是概率模型, 偶发一次"想多了"就可能执行破坏动作。所以安全设计要假设**"Agent 一定会犯错"**, 用边界兜住。

3.3.2 安全三原则 + 项目落地

原则	含义	项目已有落地
① 工具权限最小化	Agent 只能调用"完成当前任务所需的最小工具集"	白名单拦截 + Router 按意图只暴露对应工具 (<code>multi_agent_router.py</code> 的 <code>intent_node</code> 分流是雏形)
② 用户审批门 (HITL)	高风险操作必须先暂停、人工确认后才执行	✅ <code>research_agent_hitl.py</code> 的 <code>interrupt_before=["search"]</code>
③ 沙箱隔离	不可信/高风险代码在隔离环境 (Docker) 运行	✅ <code>dockerfile</code> 的资源/进程/网络隔离

3.3.3 工具风险分级

分级	例子	放行策略
● 只读工具	知识检索、网页搜索、数据库查询、文件读取	直接放行 (不改变系统状态)
● 写操作	发消息、改数据、写文件、上传	需人工审批 (HITL 拦截)
● 高危操作	执行代码、外发数据、删除文件、改权限	双重确认或直接禁止 (默认拒绝)

默认安全原则 (fail-closed) : 未登记的工具按最高风险处理——宁可误拦, 不可漏放。

3.3.4 三道防线 (安全边界架构图)





类比：白名单是"门禁卡权限"、审批门是"贵重物品人工放行"、沙箱是"即使小偷进了房间也只能在这个房间里折腾，拆不了大楼"。

4. 第 7 周面试必会问题精讲 (3 题)

Q1. 你怎么监控 Agent 性能？

参考答案要点：LangFuse 全链路追踪 + RAG 三指标自动评估，形成 feedback loop——追踪告诉"发生了什么" (token / 耗时 / 工具调用)，评估告诉"回答得好不好" (三指标对应 RAG 三环节)。

展开话术：

- "我用 LangFuse 做可观测。**追踪层**：@observe() 装饰器包住三个观测点——traced_chat (LLM 输入/输出/耗时)、traced_chat_with_tools (token 数，来自 response.usage)、traced_retrieve (检索耗时 + 命中数)，每条请求是一条 Trace，下面挂多个 Span/Generation。" (tracing.py)
- "**评估层**：evaluate_rag 用 LLM-as-Judge 打三个分——Faithfulness 忠于检索上下文吗 (生成环节)、Answer Relevance 答对题了吗 (意图对齐环节)、Context Precision 检索精准吗 (检索环节)。哪个指标掉了，就能定位是哪个环节出问题，再针对性优化 (feedback loop)。" (eval.py:256)
- "监控是为了闭环：低分样本 → 调检索 top_k / 改 prompt / 加'诚实回答'兜底，而不是拍脑袋改系统。"
- "补充我的工程细节：**双后端降级**——无外网/未配置时自动从 LangFuse 看板降级到本地 JSON 日志，追踪与评估的工程思路任何环境都保留。"

Q2. Agent 工程中 Harness 做什么？

参考答案要点：Harness = Agent 的生产基础设施层，四件套——Memory 管理 / 权限沙箱 / 任务调度 / 异常重试，让 Agent 在生产环境稳定、安全、可控运行。

展开话术：

- "① **Memory 管理**: 我用 `ConversationMemory` 做有界上下文窗口 (`max_messages=10 + _trim()`) + `SessionMemoryManager` 做会话隔离, 防止上下文无限膨胀、A 用户串到 B 用户。"
- "② **权限沙箱**: 工具白名单 (未注册一律拒绝) + 风险分级 (只读放行 / 写审批 / 高危禁止) + HITL 审批门 (`interrupt_before`) + Docker 容器隔离, 三道防线层层兜底。"
- "③ **任务调度**: 我用 `TaskQueue` (`reliability.py:79`) ——`asyncio.Queue(maxsize=10)` + 单 worker 生产-消费, 多用户请求排队不互相争抢。"
- "④ **异常重试**: `retry_with_backoff` (指数退避 + jitter) 处理 LLM 偶发失败, `run_with_timeout` 30s 超时降级, `CircuitBreaker` 处理持续故障——我在 `main.py:144` 的 `_safe_answer()` 把这三层串成调用链。"
- "一句话: Harness 是 Agent 的基础设施, 如同操作系统之于应用——没有它, Agent 只能跑在 demo 里, 一上线就翻车。"

Q3. 怎么降低 LLM 调用成本?

参考答案要点: 四招——① 模型路由 ② 缓存 ③ 精简 context ④ 控制工具调用次数。

展开话术:

- "① **模型路由**: 简单任务用便宜小模型, 复杂任务用强模型——不是所有请求都要上最强模型。"
- "② **缓存**: 相同 query 命中缓存不重复调 LLM; embedding 缓存 (文档分块后向量化结果缓存, 重建索引不重复计算)。"
- "③ **精简 context**: prompt 只传必要检索片段, 不把整库塞进去; 控制历史窗口 (我 `ConversationMemory` 的 `_trim()` 就是干这个的)。"
- "④ **工具调用次数控制**: Supervisor 的 `max_rounds` 防无效循环 (`supervisor_agent.py` 的 `rounds >= max_rounds` 强制 finish), 避免多 Agent 反复迭代烧 token。"
- "再补充可观测的视角: 结合 LangFuse 的 token 追踪 (`traced_chat_with_tools` 的 `usage`), 能精确看到哪个环节、哪个请求在烧 token, 成本优化有数据支撑而不是拍脑袋。"

5. 本周必会手写的代码 (面试手撕清单) ★

用户学习模式: **每天都要会手写当天核心代码**。以下是 Week 7 各 Day 的"必会手写能力 + 关键代码要点 + 对应文件 + 代码骨架", 面试前对着这一节默写即可。

5.1 手写 `retry_with_backoff` (指数退避 + jitter, 面试手撕最高频)

手写要点: `for attempt in range(max_retries)` 循环尝试 → 成功直接返回 → 失败且**最后一次**直接 `raise` (交给上层降级) → 否则 `delay = base * 2**attempt + random.uniform(0, 0.5)` 睡完再试。对应文件: `reliability.py:35`。

```
import time, random

def retry_with_backoff(func, max_retries=3, base_delay=1.0):
    """LLM/网络偶发失败自动重试: 退避 1s→2s→4s + 随机抖动 (jitter)。"""
    for attempt in range(max_retries):
        try:
            return func()
        except Exception as e:
            if attempt == max_retries - 1:
                # 成功: 立即返回
```

```

        raise # ★ 最后一次失败：直接抛，交给上
层降级
        delay = base_delay * (2 ** attempt) + random.uniform(0, 0.5) # ★
jitter
        time.sleep(delay)

```

面试可能追问：

- **为什么加 jitter (抖动) ?** ——若所有请求失败后按相同退避时间同时重试，会在同一时刻再次同时打向服务，形成惊群/雪崩 (thundering herd)，二次打爆下游。叠加随机量 `random.uniform(0, 0.5)` 错开各请求的重试时刻，让重试流量均匀散开，提高恢复成功率、降低二次峰值。
- **最后一次失败为什么直接抛而不继续重试?** ——`max_retries` 就是上限 (含首次)，第 `max_retries-1` 次是最后机会；重试已经耗尽仍失败，说明大概率不是偶发而是**持续故障**，再重试只会白白等待浪费时间。直接 `raise` 把异常交给上层 (如 `run_with_timeout / CircuitBreaker`) 走降级，符合“快速失败”原则。
- **哪些异常值得重试?** ——网络抖动 / 超时 / 限流 (`ConnectionError / TimeoutError / 429`) 值得重试；参数错误、认证失败等确定性错误重试也没用。工程上可用 `retryable_exceptions` 参数白名单控制 (我实现里就支持)。

5.2 手写 TaskQueue (asyncio.Queue 生产-消费者) 或 run_with_timeout (超时熔断降级)

5.2.1 TaskQueue (Day 4)

手写要点： `asyncio.Queue(maxsize=10) + submit()` 生产任务入队 + `_worker()` 后台协程逐个消费 (`get` → `await fn(*args)` → `task_done`)，哨兵对象通知 worker 退出实现优雅关闭。对应文件：`reliability.py:79`。

```

import asyncio

class TaskQueue:
    _SENTINEL = object() # 哨兵：通知 worker 退出

    def __init__(self, maxsize=10):
        self.queue = asyncio.Queue(maxsize=maxsize)
        self.worker = asyncio.create_task(self._worker())

    async def submit(self, fn, *args): # 生产：提交任务
        await self.queue.put((fn, args))

    async def _worker(self): # 消费：逐个执行
        while True:
            item = await self.queue.get()
            if item is self._SENTINEL: # 收到哨兵 → 退出
                self.queue.task_done()
                break
            fn, args = item
            try:
                await fn(*args)
            finally:
                self.queue.task_done() # ★ 标记完成，配合 join() 等待排空

```

```

async def close(self):                                # 优雅关闭：放哨兵让 worker 退出
    await self.queue.put(self._SENTINEL)
    await self.worker

async def join(self):                                  # 等待队列中所有任务执行完毕
    await self.queue.join()

```

面试可能追问：

- **为什么用 `asyncio.Queue`？** ——生产-消费者模型：多用户 `submit()` 排队，后台单 worker 逐个消费，天然串行不互相争抢；`maxsize` 限制队列上限，满则 `put` 挂起等待，自带背压 (backpressure)，不会无限堆积。
- **`task_done()` 和 `join()` 什么关系？** ——每个 `get` 出来的任务执行完必须 `task_done()` 一次，`join()` 才会计数减一；等队列里所有任务都 `task_done` 后 `join()` 返回，实现“精确等待队列排空”。少调 `task_done()` 会让 `join()` 永远等待。
- **怎么优雅停止 worker？** ——往队列放一个哨兵对象 `_SENTINEL`，worker 取到哨兵就 `break`，比强制 `cancel()` 更安全（能先把已入队任务跑完）。

5.2.2 `run_with_timeout` (超时熔断降级)

手写要点： `asyncio.wait_for(coro, timeout)` 在超时后自动取消内部协程并抛 `TimeoutError`，捕获后返回 fallback，保证调用方绝不挂死（快速失败）。对应文件：[reliability.py:137](#)。

```

import asyncio

async def run_with_timeout(coro, timeout=30, fallback="系统繁忙，请稍后再试。"):
    try:
        return await asyncio.wait_for(coro, timeout=timeout)
    except asyncio.TimeoutError:
        return fallback                                # 超时：不挂死，直接降级

```

面试可能追问：

- **超时后协程还在跑吗？** ——`asyncio.wait_for` 超时后会**取消 (cancel) **内部协程并抛 `TimeoutError`；但如果协程里是不响应取消的阻塞调用，可能仍会跑完（这就是为什么同步函数要用守护线程版 `run_with_timeout_sync`）。
- **同步函数怎么加超时？** ——同步 `func` 放进**守护线程** (`daemon=True`)，主线程 `t.join(timeout)`，超时后 `t.is_alive()` 为真则返回 fallback、后台线程继续跑但不再阻塞调用方——见 `run_with_timeout_sync()`，FastAPI 同步接口就是用它。
- **超时和熔断什么关系？** ——超时是“单次调用”的兜底（这个请求别等太久）；熔断是“整个下游”的兜底（下游持续故障就别再打）。`_safe_answer()` 里两者配合：熔断开→快速失败，没熔断→重试→再超时降级。

5.3 手写 `@observe()` LangFuse 接入 (5 行核心) + 三指标 LLM-as-Judge 打分 prompt 骨架

5.3.1 `@observe()` LangFuse 接入 (Day 1)

手写要点: `@observe()` 装饰器对现有代码**零侵入**——包住业务函数，自动记录输入/输出/耗时；若内部调 `openai` 还能自动解析 `usage` 拿 `token` 数。对应文件: `tracing.py:186` (`traced_chat`) + `tracing.py:226` (`traced_retrieve`)。

```
from langfuse.decorators import observe

@observe()                                # ① 装饰器：自动创建 Span 并挂到当前
Trace
def traced_chat(prompt: str) -> str:
    from app.llm import chat              # ② 复用已有 LLM 封装（零侵入）
    return chat(prompt)                  # ③ 输入/输出/耗时自动入 trace

@observe()                                # ④ 同样包住检索
def traced_retrieve(query: str, top_k: int = 3) -> list:
    from app.rag.hybrid_retriever import build_hybrid_retriever
    retriever = build_hybrid_retriever("data/employee_policy.txt")
    return retriever.retrieve(query, top_k=top_k) # ⑤ 检索耗时/命中数自动入 trace
```

面试可能追问:

- `@observe()` 的“隐藏工作”是什么？——① 创建 `Span/Generation` 并**自动取名**（默认函数名）挂到当前 `Trace`；② 捕获**入参与返回值**；③ 记录**耗时**；④ 若内部调 `openai` 自动解析 `usage` → **`token` 数 / 成本**。你在业务代码里看不到这些，这就是“装饰器 + 零侵入”的价值。
- `traced_chat` 为什么拿不到 `token` 数？——原 `app.llm.chat` 返回字符串（`response.usage` 在 `response` 上而非 `message` 上），原函数未透传 `response`。为拿到 `usage` 又不动 `llm.py`，我写了 `traced_chat_with_tools` (`tracing.py:200`) ——用相同 `OpenAI` 配置重建调用，返回 `{"message": ..., "usage": response.usage}`，语义不变且 `token` 进追踪。
- 无外网时怎么办？——`_langfuse_configured()` 探测失败自动降级为自写 `_local_observe()` (`tracing.py:112`)：打印 `span.start / span.end` 的 `JSON` 日志（含 `elapsed_ms / usage / 异常`），接口与 `LangFuse` 完全一致。

5.3.2 三指标 LLM-as-Judge 打分 prompt 骨架 (Day 2)

手写要点: 三个 Judge prompt 各约束“只输出 0 或 1”→ `_llm_judge` 调 `chat` 并严格解析 → 失败降级规则兜底。对应文件: `eval.py:58` (`FAITHFULNESS_PROMPT`) + `eval.py:71` (`ANSWER_RELEVANCE_PROMPT`) + `eval.py:84` (`CONTEXT_PRECISION_PROMPT`)。

```
from app.llm import chat

# ① Faithfulness (生成环节)：回答是否忠于 context
FAITHFULNESS_PROMPT = """你是 RAG 回答质量评估员。请判断"回答"是否忠于"上下文"。
规则：若回答中的每个事实都能在上下文中找到依据，输出 1；否则输出 0。
上下文: {context}
回答: {answer}
只输出 0 或 1，不要输出任何其他内容。"""

# ② Answer Relevance (意图对齐环节)：不看 context，只看是否对题
ANSWER_RELEVANCE_PROMPT = """你是 RAG 回答质量评估员。请判断"回答"是否真的回答了"问
```

题"。

规则：不看任何检索上下文，只比较"问题"与"回答"。若回答切题、直接回应，输出 **1**；若答非所问，输出 **0**。

问题: {question}

回答: {answer}

只输出 **0** 或 **1**，不要输出任何其他内容。"""

③ Context Precision (检索环节)：正确答案是否靠前

CONTEXT_PRECISION_PROMPT = """你是 RAG 回答质量评估员。请判断"检索到的上下文"是否精准且不多余。

规则：根据"问题"判断—包含正确答案的片段是否出现在检索结果前列。靠前且不冗余输出 **1**；否则输出 **0**。

问题: {question}

检索结果（按序）：

{chunks}

只输出 **0** 或 **1**，不要输出任何其他内容。"""

```
def score_faithfulness(context: str, answer: str) -> int:
    result = chat(FAITHFULNESS_PROMPT.format(context=context, answer=answer))
    return 1 if "1" in result else 0    # 严格解析：只输出 0/1，宽松兜底取首个数字
```

写回 LangFuse (feedback loop)

from langfuse import Langfuse

langfuse = Langfuse()

langfuse.score(name="faithfulness", trace_id=trace_id, value=score_faithfulness(context, answer))

面试可能追问：

- **为什么用"只输出 0 或 1"？**——二分类问题让 LLM 只输出一个数字，**解析最简单、最稳**，避免自由文本难解析；我 `_llm_judge` 还做了宽松兜底（取文本中首个数字），LLM 输出不规范也不崩。
- **LLM-as-Judge 失败/超时怎么办？**——降级到**规则兜底**（通道 B）：中文 **2-gram**（免分词）关键词覆盖率近似打分——`_rule_faithfulness` 看回答 2-gram 被 context 覆盖的比例 ≥ 0.5 判 1 (`eval.py:122`)，`_rule_answer_relevance` 看问题信息被回答覆盖比例 (`eval.py:136`)，`_rule_context_precision` 看正确答案是否在首位 (`eval.py:151`)。双通道保证任何环境不崩。
- **为什么三指标要对应 RAG 三环节？**——排障定位：Faithfulness 掉 → 生成环节 LLM 硬编，需"诚实回答"兜底；Answer Relevance 掉 → 意图对齐环节答非所问；Context Precision 掉 → 检索环节结果不准，调 `top_k` / 重排。**先定位再优化**，不是拍脑袋改系统。

5.4 手写 `CircuitBreaker` 状态机 + 工具白名单拦截（可选加分项）

5.4.1 `CircuitBreaker` 状态机（加分项）

手写要点：三状态 `closed` / `open` / `half_open` + 失败计数 + 冷却时间 + 半开探测。对应文件：

`reliability.py:182`。

```
import time
```

```
class CircuitBreaker:
```

```
    STATE_CLOSED, STATE_OPEN, STATE_HALF_OPEN = "closed", "open", "half_open"
```

```

def __init__(self, failure_threshold=3, cooldown=5.0, fallback="服务暂时不可
用"):
    self.failure_threshold = failure_threshold
    self.cooldown = cooldown
    self.fallback = fallback
    self._failures = 0
    self._state = self.STATE_CLOSED
    self._opened_at = 0.0

def call(self, func, *args, **kwargs):
    if self._state == self.STATE_OPEN:
        if time.time() - self._opened_at < self.cooldown:
            return self.fallback          # 冷却期: 快速失败, 不调底层
        self._state = self.STATE_HALF_OPEN # 冷却结束: 半开, 放探测请求
    try:
        result = func(*args, **kwargs)
    except Exception as e:
        self._failures += 1
        if self._state == self.STATE_HALF_OPEN:
            self._state = self.STATE_OPEN    # 探测失败: 立即回 open
            self._opened_at = time.time()
        elif self._failures >= self.failure_threshold:
            self._state = self.STATE_OPEN    # 连续失败达阈值: 打开
            self._opened_at = time.time()
        raise e                             # 如实上抛, 交给上层重试/降级
    else:
        self._failures = 0                  # 成功: 复位
        self._state = self.STATE_CLOSED
        return result

```

面试可能追问:

- **熔断和重试怎么配合?** ——重试管"偶发失败" (试几次可能就好), 熔断管"持续故障" (下游挂了别反复打)。_safe_answer() 里由内到外: CircuitBreaker.call(重试(fn))——熔断 open 期间不消耗真实调用直接快速失败, 没熔断才走重试再超时降级 (main.py:144)。
- **半开状态解决什么问题?** ——熔断打开后下游可能已恢复, 但不能立刻全量放行 (会再次打爆)。half_open 只放一个探测请求: 成功→复位 closed, 失败→立即回 open 重新冷却——用小流量试探, 平滑恢复。

5.4.2 工具白名单拦截 + 风险分级 (Day 5 概念核心)

手写要点: 白名单放行 + 风险分级 (只读放行 / 写审批 / 高危禁止), **未登记按最高风险处理 (fail-closed)**。对应文件: security_notes.md (RISK_LEVEL + call_tool_guarded)。

```

# ① 工具权限最小化: 白名单注册, 未注册工具一律拒绝
ALLOWED_TOOLS = {"knowledge_search", "safe_web_search"}

def call_tool_safe(name: str, args: dict):
    if name not in ALLOWED_TOOLS:

```

```

        raise PermissionError(f"工具 {name} 不在白名单，已拦截")
    return execute(name, args)

# ② 工具风险分级：只读放行 / 写审批 / 高危禁止 (fail-closed)
RISK_LEVEL = {
    "knowledge_search": "read",
    "safe_web_search": "read",
    "send_message":     "write",
    "delete_data":      "danger",
}

def call_tool_guarded(name, args, approve=None):
    level = RISK_LEVEL.get(name, "danger") # 未登记 → 按最高风险（默认拒绝）
    if level == "read":
        return execute(name, args) # 只读直接放行
    if level == "write":
        return execute(name, args) if approve() else "已拦截：该操作需要用户确认"
    raise PermissionError(f"高危操作 {name} 已禁止")

```

面试可能追问：

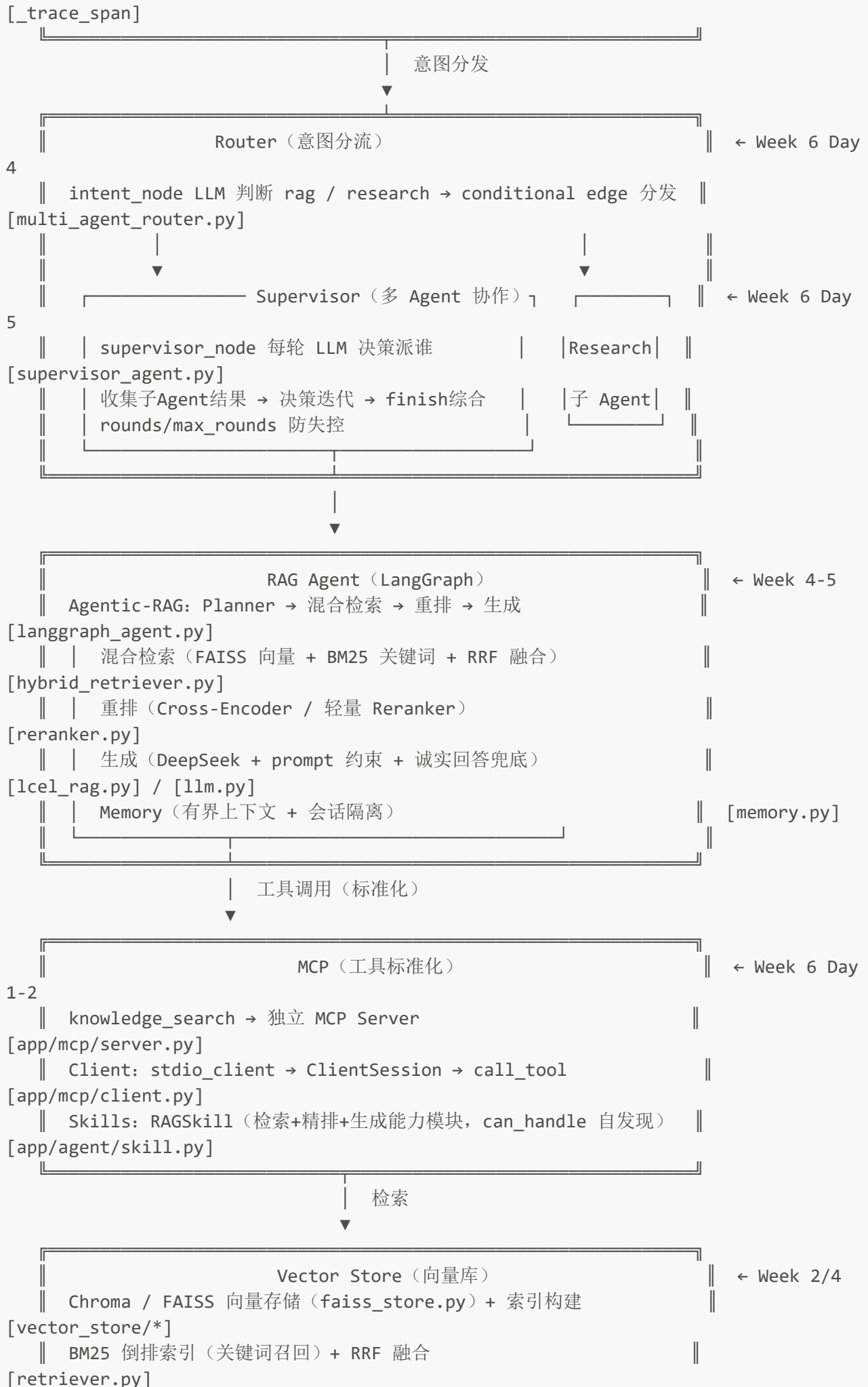
- **为什么未登记的工具按 danger 处理？** ——**fail-closed（默认拒绝）**原则：安全设计假设"Agent 一定会犯错"，宁可误拦也不可漏放；新工具必须先登记、评估风险级别才允许调用，避免"没进白名单的工具悄悄能用"的安全漏洞。
- **写操作怎么人工确认？** ——HITL 审批门：`approve()` 是人工确认回调，无人确认 → 返回"需要确认"并拦截。真实项目里对应 LangGraph 的 `interrupt_before=["search"] + update_state + invoke(None)` (`research_agent_hitl.py`)。

总结：这 5 块 (`retry_with_backoff` / `TaskQueue+run_with_timeout` / `@observe()` + 三指标 `Judge` / `CircuitBreaker` + 白名单拦截) 是 Week 7 面试手撕的***最小可写集合***——能默写它们，就能讲清"可观测 → 可靠性 → 安全 → 整合"的完整工程化闭环。

6. 完整架构图：企业级 Agent 系统全链路 (Week 7 终极面试作品)

这张图把 8 周所有技术栈串成一条线：从"手写 RAG"到"企业级 Agent 系统"。面试时从外到内一层层讲，每层标注对应的 Week 产物文件。





|| 数据源: data/employee_policy.txt → 分块 → 向量化 → 入库 ||
[build_index.py]

横切层（贯穿所有请求，Week 7 三件套）：

- ① 可观测（Week 7 Day 1-2）LangFuse 追踪 + 评估

traced_chat / traced_retrieve / evaluate_rag 三指标

LangFuse ↔ 本地 JSON 日志 双后端透明切换

文件: [app/observability/tracing.py] · [app/observability/eval.py]
- ② 可靠性（Week 7 Day 3-4）Harness 四件套

retry_with_backoff（指数退避+jitter）/ TaskQueue（调度）

run_with_timeout（30s 超时）/ CircuitBreaker（熔断）

Memory（已有）/ 权限沙箱（Day 5）

文件: [app/agent/reliability.py]
- ③ 安全（Week 7 Day 5）权限沙箱三防线

白名单+风险分级（最小化）→ HITL 审批门 → Docker 隔离

文件: [research_agent_hitl.py] · [dockerfile] · [security_notes.md]

架构图一句话背法（从外到内）：

"FastAPI 网关（路由 + `_safe_answer` 可靠性包装）→ Router 意图分流 → Supervisor 多 Agent 协作 → RAG Agent（LangGraph + 混合检索 + 重排 + Memory）→ MCP 工具标准化 → Vector Store 向量库；整条链路被三层横切：LangFuse 可观测（追踪+评估）、Harness 可靠性（重试/队列/超时/熔断）、权限沙箱（白名单/HITL/Docker）。"

架构图分层对照表（每层对应 Week 产物）：

层	职责	对应 Week 产物文件
FastAPI 网关	HTTP 入口 + 12 路由 + 统一可靠性/可观测接入	app/main.py (Week 7)
Router	意图分流（rag / research 一次分发）	app/agent/multi_agent_router.py (Week 6)
Supervisor	多 Agent 可迭代协作 + 收集 + 综合 + 防失控	app/agent/supervisor_agent.py (Week 6)
RAG Agent	LangGraph + 混合检索 + 重排 + 生成 + Memory	app/agent/langgraph_agent.py · app/rag/hybrid_retriever.py · app/memory/memory.py (Week 4-5)
MCP	工具标准化 (Server/Client) + Skills 能力模块	app/mcp/server.py · app/agent/skill.py (Week 6)

层	职责	对应 Week 产物文件
Vector Store	向量检索 + BM25 关键词 + 索引构建	app/rag/vector_store/faiss_store.py · app/rag/build_index.py (Week 2/4)
横切·可观测	LangFuse 追踪 (3 观测点) + 评估 (3 指标)	app/observability/tracing.py · app/observability/eval.py (Week 7)
横切·可靠性	重试/队列/超时/熔断 (Harness 四件套)	app/agent/reliability.py (Week 7)
横切·安全	白名单 + HITL + Docker 沙箱 (三道防线)	research_agent_hitl.py · dockerfile (Week 5 概念)

7. Git 提交清单

7.1 本周新增/修改文件清单 (相对路径)

文档类:

- week7plan.md (周计划)
- langfuse_notes.md (Day 1-2 LangFuse 追踪 + 评估笔记: 三层观测点 + @observe() + 降级方案)
- harness_notes.md (Day 3 Harness 四件套概念 + 组件对照表)
- security_notes.md (Day 5 安全三原则 + 工具风险分级 + 三道防线)
- week7_review.md (本文件)

可观测模块:

- app/observability/__init__.py
- app/observability/tracing.py (Day 1: traced_chat / traced_chat_with_tools / traced_retrieve + 双后端)
- app/observability/eval.py (Day 2: 三指标 + LLM-as-Judge + 规则兜底 + 写回 LangFuse)

可靠性模块:

- app/agent/reliability.py (Day 4: retry_with_backoff / TaskQueue / run_with_timeout / run_with_timeout_sync / CircuitBreaker)
- test_reliability.py (Day 4: 20 项测试全 PASS)

项目整合:

- app/main.py (Day 6: 12 条路由 + _safe_answer 可靠性调用链 + _trace_span 可观测接入点)
- README.md (Day 6: 系统架构 + 接口清单 + 可靠性/可观测说明更新)

7.2 建议的 git commit 命令序列 (分组提交)

```
# ① 计划文档
git add week7plan.md
git commit -m "Week7: 周计划—工程化闭环 (可观测 + 可靠性 + 安全 → 项目整合)"

# ② LangFuse 可观测 (Day 1-2): 追踪 + 评估
```

```
git add langfuse_notes.md
git commit -m "Week7 Day1: LangFuse 概念笔记（三层观测点 + @observe 零侵入 + 降级方案）"

git add app/observability/__init__.py app/observability/tracing.py
git commit -m "Week7 Day1: traced_chat/traced_retrieve 接入 LangFuse 追踪（LangFuse ↔ 本地 JSON 双后端透明切换）"

git add app/observability/eval.py
git commit -m "Week7 Day2: 三指标评估
score_faithfulness/answer_relevance/context_precision（LLM-as-Judge + 中文2-gram规则兜底）"

# ③ Harness 概念（Day 3）
git add harness_notes.md
git commit -m "Week7 Day3: Harness 四件套概念笔记（Memory/沙箱/调度/重试）"

# ④ 可靠性实战（Day 4）：重试 + 队列 + 超时 + 熔断
git add app/agent/reliability.py test_reliability.py
git commit -m "Week7 Day4: reliability.py—指数退避重试(jitter)/TaskQueue/超时熔断降级/CircuitBreaker, 20项测试全PASS"

# ⑤ 安全概念（Day 5）
git add security_notes.md
git commit -m "Week7 Day5: Agent 安全三原则笔记（最小化/HITL/沙箱）+ 工具风险分级"

# ⑥ 项目整合（Day 6）：Week 4-7 全链路进 main.py
git add app/main.py README.md
git commit -m "Week7 Day6: 整合—rag/supervisor /rag/router 接口 + _safe_answer 可靠性调用链 + _trace_span 可观测接入点"

# ⑦ 周回顾收尾（Day 7）
git add week7_review.md
git commit -m "Week7 Day7: 周回顾—工程化闭环 + 企业级 Agent 系统全链路架构图"
```

8. 自检清单（面试前逐条勾选）

8.1 三技能验收

- ☐ 能手画“企业级 Agent 系统全链路”架构图（FastAPI → Router → Supervisor → RAG Agent → MCP → VectorStore + 三层横切），每层标注 Week 产物文件
- ☐ 能手画 LangFuse 追踪链路图（用户请求 → LLM → 检索 → 工具 → LLM → 回答，标注 3 观测点），并讲清双后端降级
- ☐ 能手画 Harness 四件套架构图 + 熔断器状态机（closed → open → half_open → closed）
- ☐ 能手画 Agent 安全边界图（三道防线：白名单 → HITL → Docker 沙箱）

8.2 三道面试题（能否不看笔记口头讲清，每条 30 秒内）

- ☐ **怎么监控 Agent 性能？** ——LangFuse 全链路追踪（token/耗时/工具）+ 三指标评估（Faithfulness/Answer Relevance/Context Precision 对应 RAG 三环节）+ feedback loop

- ☐ **Agent 工程中 Harness 做什么？** ——四件套（Memory 管理 / 权限沙箱 / 任务调度 / 异常重试），每件套举项目真实代码位置
- ☐ **怎么降低 LLM 调用成本？** ——模型路由 / 缓存 / 精简 context / 控制工具调用次数（max_rounds 防无效循环）

8.3 手写代码（能否对着骨架默写，并答出追问）

- ☐ `retry_with_backoff`（指数退避 + jitter）——答出“为什么加 jitter（防惊群）”、“最后一次为什么直接抛（快速失败交给上层降级）”
- ☐ `TaskQueue`（`asyncio.Queue` 生产-消费者）——答出“为什么用 Queue（串行不争抢 + 背压）”、“task_done/join 关系”、“哨兵优雅关闭”
- ☐ `run_with_timeout`（超时熔断降级）——答出“wait_for 超时取消”、“同步版守护线程不阻塞”
- ☐ `@observe()` LangFuse 接入（5 行核心）——答出“装饰器隐藏工作（自动记录输入输出/token/耗时）”、“traced_chat 为什么拿不到 usage”
- ☐ 三指标 LLM-as-Judge 打分 prompt 骨架——答出“为什么只输出 0/1”、“LLM 失败降级规则兜底（中文 2-gram）”
- ☐（加分）`CircuitBreaker` 状态机 + 工具白名单拦截——答出“熔断 vs 重试”、“fail-closed 默认拒绝”

8.4 三指标 ↔ RAG 三环节（能讲清对应关系）

- ☐ Faithfulness（生成环节）掉分 → LLM 硬编 → “诚实回答”兜底
- ☐ Answer Relevance（意图对齐环节）掉分 → 答非所问 → 查 query/prompt
- ☐ Context Precision（检索环节）掉分 → 检索不准/冗余 → 调 top_k / 加重排

最后自我评估：能不看笔记，把上面“3 道面试题 + 3 个手撕核心代码（retry + queue + 超时）+ 完整架构图 + 三指标对应关系”**口头讲一遍**（对着镜子或录音），每条 30 秒内讲完，就算过关。

9. 本周学习闭环：从“能跑”到“能上生产”的一句话总结

前 6 周你证明了“我能写 Agent”；第 7 周你证明了“我能驾驭 Agent”——**监控它（可观测）、扛住它（可靠性）、保护它（安全）**。面试官最想听的正是这三点：不是“我调了 API”，而是“**我监控了它、重试了它、保护了它**”。这张企业级架构图，就是你 8 周学习的终极面试作品。